



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 05

NOMBRE COMPLETO: ALFONSO D'HERNÁN RODRÍGUEZ
ZULUAGA

N° de Cuenta: 321561556

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 05

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 29 de marzo del 2026

CALIFICACIÓN: _____

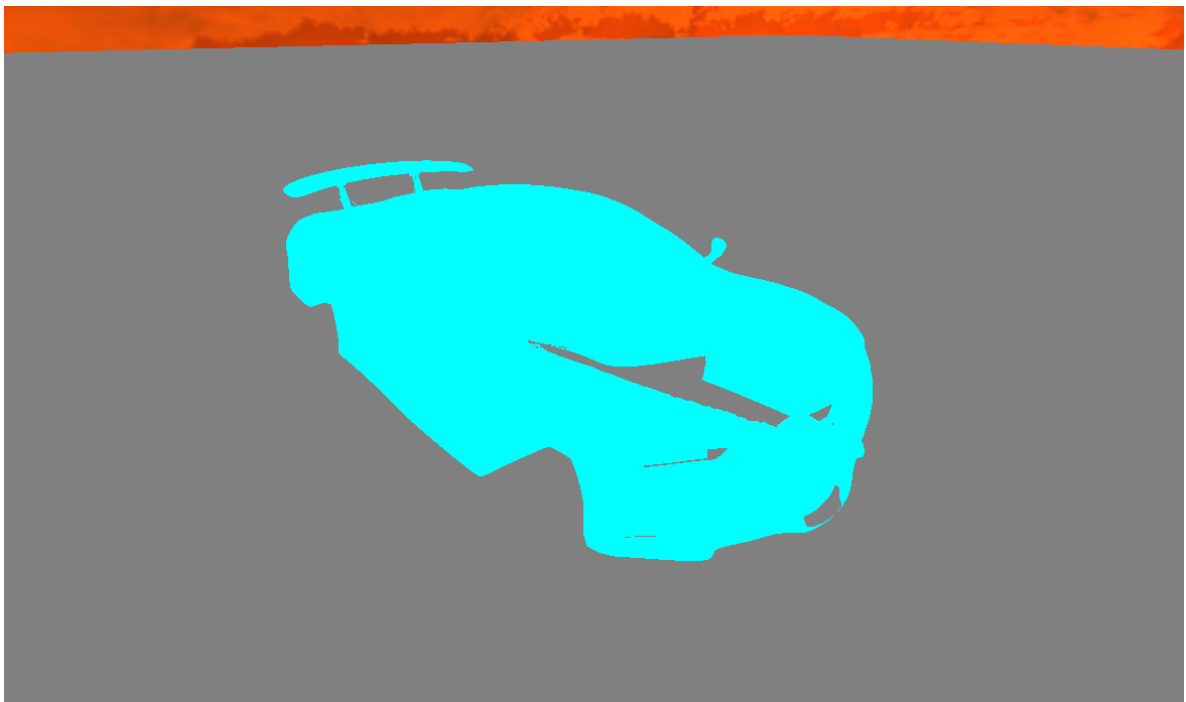
1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

a. Importación del coche

Código

```
Carro_M = Model();  
Carro_M.LoadModel("Models/Buggati.obj");  
  
color = glm::vec3(0.0f, 1.0f, 1.0f); //carro cyan  
model = glm::mat4(1.0);  
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));  
model = glm::translate(model, glm::vec3(0.0f, 0.0f, mainWindow.getDesplazamiento()));  
model = glm::translate(model, glm::vec3(0.0f, -2.0f, -7.5f));  
modelaux = model;  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Carro_M.RenderModel();  
model = modelaux;
```

Escena



b. Importación de las llantas y giro

Código

Window.h

```
GLfloat getRuedas() { return ruedas; }

GLfloat desplazamiento = 0, ruedas = 0, cofre = 0;
```

Window.cpp

```
,
if (key == GLFW_KEY_X)
{
    theWindow->ruedas += 10.0;
}
```

Main

```
color = glm::vec3(0.0f, 0.0f, 0.0f); //llanta color negro
model = glm::translate(model, glm::vec3(-4.0f, 1.75f, 3.1f));
model = glm::rotate(model, glm::radians(mainWindow.getRuedas()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(10.0f), glm::vec3(0.0f, 1.0f, 0.0f));
//model = glm::scale(model, glm::vec3(30.0f, 30.0f, 30.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_PDD.RenderModel();
model = modelaux;

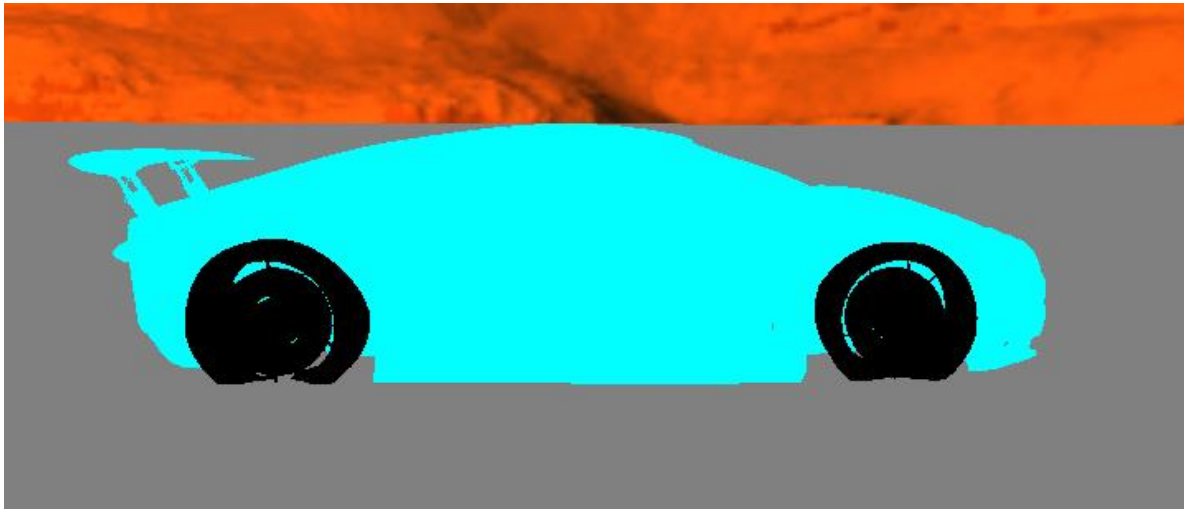
color = glm::vec3(0.0f, 0.0f, 0.0f); //llanta color negro
model = glm::translate(model, glm::vec3(1.3f, 1.75f, 2.1f));
model = glm::rotate(model, glm::radians(mainWindow.getRuedas()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(10.0f), glm::vec3(0.0f, 1.0f, 0.0f));
//model = glm::scale(model, glm::vec3(30.0f, 30.0f, 30.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_PDI.RenderModel();
model = modelaux;

color = glm::vec3(0.0f, 0.0f, 0.0f); //llanta color negro
model = glm::translate(model, glm::vec3(-4.5f, 1.75f, -5.0f));
model = glm::rotate(model, glm::radians(mainWindow.getRuedas()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(10.0f), glm::vec3(0.0f, 1.0f, 0.0f));
//model = glm::scale(model, glm::vec3(30.0f, 30.0f, 30.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_PTD.RenderModel();
model = modelaux;

color = glm::vec3(0.0f, 0.0f, 0.0f); //llanta color negro
model = glm::translate(model, glm::vec3(0.0f, 1.75f, -5.4f));
model = glm::rotate(model, glm::radians(mainWindow.getRuedas()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(10.0f), glm::vec3(0.0f, 1.0f, 0.0f));

//model = glm::scale(model, glm::vec3(30.0f, 30.0f, 30.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_PTI.RenderModel();
model = modelaux;
```

Escena



c. Importación del cofre y giro

Código

Window.h

```
GLfloat desplazamiento = 0, ruedas = 0, cofre = 0;  
GLfloat flag_cofre = 0;  
  
GLfloat getCofre() { return cofre; }
```

Window.cpp

```

if (key == GLFW_KEY_V)
{
    if (theWindow->flag_cofre == 0)
    {
        theWindow->cofre += 5;
    }
    else
    {
        theWindow->cofre -= 5;

        if (theWindow->cofre == 90) {
            theWindow->flag_cofre = 1;
        }
        else if (theWindow->cofre == 0) {
            theWindow->flag_cofre = 0;
        }
    }
}

```

Main

```

color = glm::vec3(1.0f, 0.0f, 0.0f); //cofre rojo
model = glm::translate(model, glm::vec3(-1.35f, 2.85f, 3.5f));
model = glm::rotate(model, glm::radians(-mainWindow.getCofre()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(-10.0f), glm::vec3(0.0f, 0.0f, 1.0f));
//model = glm::scale(model, glm::vec3(30.0f, 30.0f, 30.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_Cofre.RenderModel();

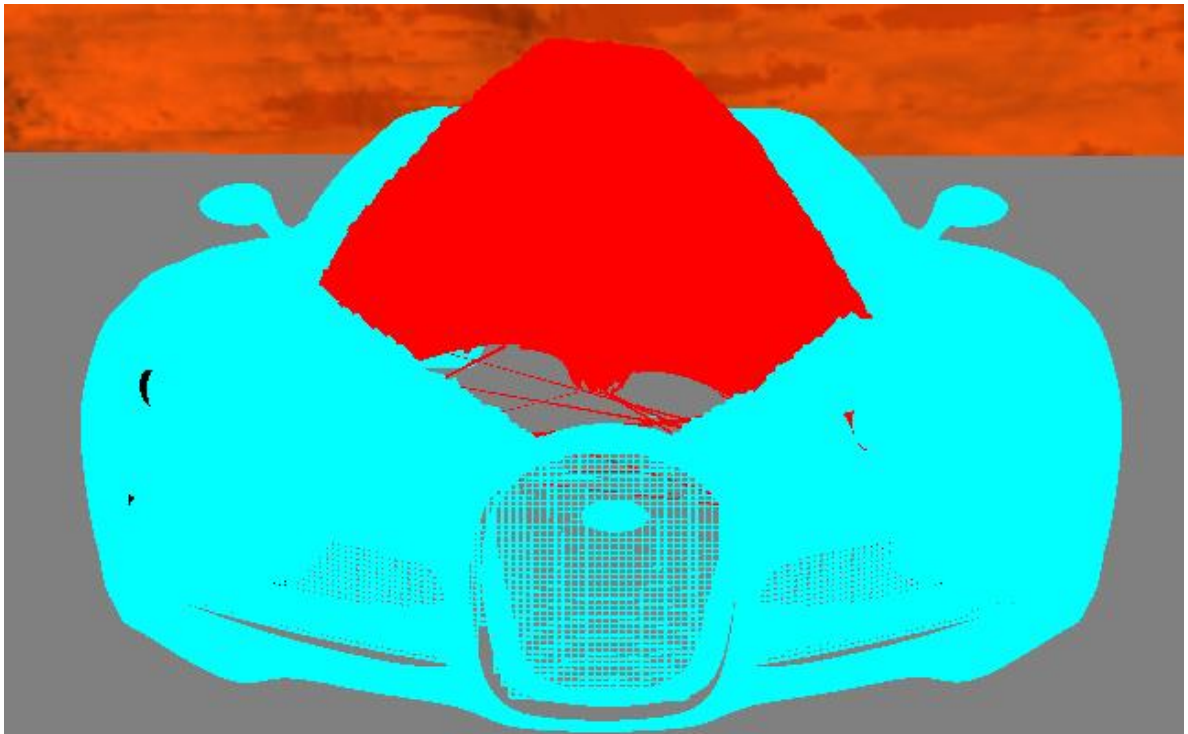
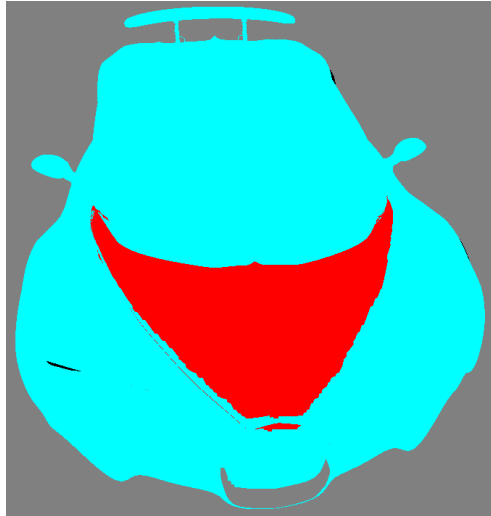
```

```

Carro_Cofre = Model();
Carro_Cofre.LoadModel("Models/Cofre.obj");

```

Escena



d. Movimiento del coche.

Código

Window.h

```
GLfloat desplazamiento = 0;  
GLfloat getDesplazamiento() { return desplazamiento; }
```

Window.cpp

```

if (key == GLFW_KEY_Z)
{
    theWindow->desplazamiento -= 5.0;
}
if (key == GLFW_KEY_X)
{
    theWindow->ruedas += 10.0;
}
if (key == GLFW_KEY_C)
{
    theWindow->desplazamiento += 5.0;
}

```

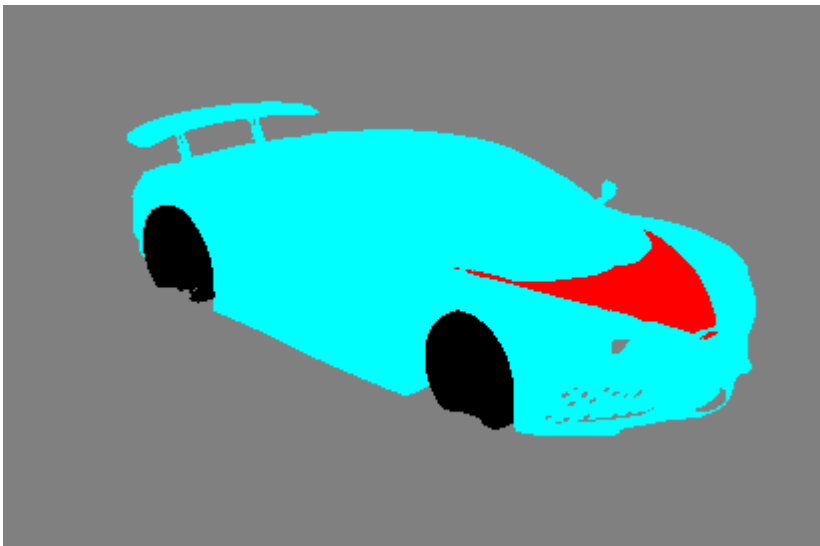
Main

```

color = glm::vec3(0.0f, 1.0f, 1.0f); //carro cyan
model = glm::mat4(1.0);
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
model = glm::translate(model, glm::vec3(0.0f, 0.0f, mainWindow.getDesplazamiento()));
model = glm::translate(model, glm::vec3(0.0f, -2.0f, -7.5f));

```

Escena





2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

- Tuve un problema bastante grande a la hora de primero querer importar el modelo. El problema en cuestión radicó en que habían partes del modelo que simplemente no generaban normales. Esto, a la hora de ejecutar, causaba que se leyeran apuntadores nulos en vez de normales, así llegando a una pausa total del programa. Debo admitir que tuve que recibir ayuda de IA, pues investigué ampliamente y no encontré algo que me ayudara. Simplemente tuve que quitar las partes de mi modelo que no tenían normales.

- También entendí la parte de Set to Origin, y cómo es que esto afectaba a los modelos exportados, así como el Apply Transform que se ve en Blender a la hora de exportar. Esto me permitió superar la parte de no poder rotar debidamente las ruedas y demás.

3.- Conclusión:

La explicación a nivel clase la encontré un poco rápida, creo que hubieron detalles que se pinclearon de manera demasiado rápida para registrarla debidamente, pero en cualquier otro caso estuvo bien.

La práctica me pareció fantástica. Muy entendible, los problemas que resolví los resolví con un entendimiento pleno de la solución y hacer la implementación fue muy divertido en todos los aspectos, salvo la cuestión de acomodar los modelos después de la jerarquización.

Viendo cómo el objetivo de la presente práctica era importar modelos, separar diferentes cantidades de polígonos para después poder jerarquizar los modelos y tener animaciones presentes, así como poder aplicar los conceptos anteriormente vistos en clase a los modelos obtenidos de Blender, considero la práctica completamente concluida.

Bibliografía en formato APA

Román Guadarrama, J. R. (2026). Manual de prácticas: (Práctica No. 03).

Universidad Nacional Autónoma de México, Facultad de Ingeniería, División de
Ingeniería Eléctrica, Departamento de Computación.